A Prose-Tightened Mathematical Companion To Fixed Sparse-Grid Quadrature Filtering And Its Analytical Gradient

Chak Wong

2026-06-12

Abstract

This note develops fixed sparse-grid quadrature filtering as a mathematically explicit high-dimensional filtering proposal for the regime where exact nonlinear filtering is unavailable but a deterministic Gaussian-surrogate value-and-gradient target is still scientifically useful. Starting from the sparse-grid quadrature nonlinear filter of Jia, Xin, and Cheng [1], the note reconstructs the Gaussian approximation and sparse-grid formulas in source order, then defines a fixed approximate likelihood scalar together with an analytical derivative of that same scalar on a declared branch. The document is written for both review and implementation: it states the exact filtering target, the narrower carried Gaussian surrogate, the fixed cloud-construction rule, the per-step value recursion, the same-scalar branch contract, the analytical gradient recursion, and the finite-difference validity rule. The central limitation is explicit. The reported scalar is not the exact nonlinear filtering likelihood, and the method does not preserve general non-Gaussian posterior shape; it is a deterministic Gaussian-surrogate lane whose usefulness depends on whether one carried Gaussian and a fixed sparse-grid moment engine are adequate for the intended regime.

1 Introduction And Scope

High-dimensional nonlinear filtering forces a choice of computational object. The exact Bayesian filter propagates a full conditional law through time, but in nonlinear models that law usually does not remain available in a finite closed form. In low dimension one may still tolerate heavy deterministic quadrature or a richer non-Gaussian representation. In high dimension, however, both cost and structural complexity escalate quickly. This note studies one narrower lane suited to that regime: a fixed sparse-grid quadrature filter that carries a single Gaussian surrogate, evaluates the required nonlinear moments by a fixed deterministic cloud in standardized Gaussian coordinates, and differentiates the same declared approximate scalar on a frozen computational branch.

The note is intended as a chapter-like companion to the monograph rather than as a survey or a software manual. Its task is to make one deterministic Gaussian-surrogate lane precise enough to read, implement, audit, and compare. The argument therefore moves from the exact filtering target, to the Gaussian projection that replaces it, to the sparse-grid cloud that supplies the required moments, to the fixed scalar and its same-scalar derivative, and finally to the verification and positioning obligations that bound the method scientifically.

The central limitation should be fixed at the outset. FixedSGQF does not claim exact nonlinear filtering, exact nonlinear likelihood evaluation, or faithful preservation of arbitrary multimodality, skewness, or high-order global interaction structure. Its purpose is narrower: to ask whether one carried Gaussian, together with a fixed sparse-grid moment engine and a fixed-branch value-and-

gradient contract, can provide a coherent and scientifically useful approximate likelihood lane in the regime under study.

Background assumed. The reader is assumed to know multivariate Gaussian notation, state-space-model notation, Jacobian-style derivative notation, and the use of a Cholesky factor for a positive-definite covariance matrix. Sparse-grid merging, same-scalar branch identities, and fixed-cloud derivative obligations are defined where they first become necessary.

Contents

1.1 Problem Setting And The FixedSGQF Lane

The scientific problem is high-dimensional nonlinear filtering. At time t, the exact Bayesian filter propagates and updates the full conditional law $p_\theta(x_t \mid y_{1:t})$. In nonlinear models that law is generally not available in a finite-dimensional closed form, and in high dimension it becomes unrealistic to treat exact function-valued propagation as the practical computational object. FixedSGQF therefore does not attempt to rescue exactness by notation. It selects one deterministic approximation lane whose strengths and boundaries can both be stated explicitly.

That lane is fixed sparse-grid quadrature filtering. The carried state is a single Gaussian surrogate $\mathcal{N}(m_t, P_t)$. The nonlinear moments needed by the Gaussian projection are approximated with a deterministic sparse-grid rule in standardized Gaussian coordinates. The cloud, duplicate-merge policy, covariance-factor branch, and veto rules are fixed before value and gradient evaluation so that the reported derivative differentiates the same scalar that the reported value routine computes. The gain from that design is transparency: each approximation stage is named, reproducible, and auditable. The cost is that a single Gaussian carried object cannot preserve general non-Gaussian posterior geometry even when the quadrature rule is itself accurate for the moments it is asked to approximate.

FixedSGQF should therefore not be read as a weaker version of a richer density-approximation lane such as squared tensor-train filtering. The present note is about the deterministic Gaussian-surrogate lane with a fixed cloud and a same-scalar analytical gradient.

1.2 The Selected Approximation Lane In One Page

The lane studied here is defined by three deliberate approximations.

1. **Gaussian carried state.** Instead of carrying the full filtering law, the method carries one Gaussian summary $\mathcal{N}(m_t, P_t)$.

2. **Sparse-grid moment closure.** The nonlinear Gaussian moments that define the prediction and observation projection are approximated by a fixed sparse-grid rule in standardized coordinates.

3. **Fixed-branch differentiation.** The reported gradient is the derivative of a declared deterministic surrogate scalar on a frozen branch, not the derivative of a live adaptive algorithm.

The resulting accumulated scalar is

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta), \tag{1.1}$$

where each increment is the Gaussian innovation log likelihood implied by the sparse-grid moment approximation. The exact object, the carried surrogate, and the accumulated scalar are therefore different in kind:

$$\begin{aligned} \text{exact object:} \quad & p_\theta(x_t \mid y_{1:t}), \\ \text{carried surrogate:} \quad & \mathcal{N}(m_t, P_t), \\ \text{declared deterministic scalar:} \quad & \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta). \end{aligned} \tag{1.2}$$

This distinction governs everything that follows. The note does not return to it repeatedly because it changes; it returns to it only when a later construction needs the distinction in a sharper form.

1.3 Approximation Hierarchy And Reading Guide

The note unfolds in four large movements. First, the exact filtering recursion and the Gaussian projection that replaces it are fixed formally. Second, the sparse-grid construction is built from low-dimensional examples through one-dimensional rules, tensor products, active index sets, and merged clouds. Third, that fixed cloud is inserted into the filtering value recursion and then into the same-scalar derivative recursion. Fourth, the resulting lane is verified, positioned against neighboring deterministic Gaussian alternatives, and bounded by explicit non-claims.

The main coordinates are:

$$\xi \in \mathbb{R}^{n_x}, \qquad \xi \sim \mathcal{N}(0, I_{n_x}) \qquad \text{standard quadrature coordinate,} \tag{1.3}$$

$$x = m + C\xi, \quad CC^\top = P \qquad \text{physical state coordinate under a carried Gaussian,} \tag{1.4}$$

$$a = f_\theta(x), \qquad \text{transition image used for prediction,} \tag{1.5}$$

$$z = h_\theta(x), \qquad \text{observation image used for the innovation.} \tag{1.6}$$

The sparse grid is built once in ξ-space and then placed into physical state space through the current Gaussian factor. That one sentence is worth retaining before the technical development begins, because it is the bridge between the cloud-construction chapter and the later value and gradient recursions.

2 Exact Filtering And Gaussian Projection

Use the additive-noise state-space model

$$X_t = f_\theta(X_{t-1}) + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{2.1}$$

$$Y_t = h_\theta(X_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \tag{2.2}$$

with $X_t \in \mathbb{R}^{n_x}$, $Y_t \in \mathbb{R}^{n_y}$, and parameter $\theta \in \mathbb{R}^p$. Throughout this note, the process noises η_t are assumed zero mean, mutually independent of the observation noises ε_s for all relevant time indices, independent across time within the process-noise family, and independent of the past latent states and observations conditional on the model. Likewise, the observation noises ε_t are assumed zero mean, independent across time within the observation-noise family, and independent of the latent states conditional on the declared additive observation model. These assumptions are exactly what let the filtering factorization and the later covariance identities use Q_θ and R_θ as the declared additive noise covariances. The exact Bayesian prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) \, \mathrm{d}x_{t-1}, \tag{2.3}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{\int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) \, \mathrm{d}x_t}. \tag{2.4}$$

These are the conceptual filtering equations used by Gaussian approximation filters in Jia, Xin, and Cheng [1, Section 2]. In nonlinear models, the integrals in (2.3) and (2.4) usually do not close in finite-dimensional form. Fixed SGQF replaces those exact densities by a Gaussian moment projection.

2.1 Report-Wide Object Map And Implementation Conventions

The report uses the following principal objects throughout the value and gradient paths. An implementation worker should treat this table as the core object inventory for the note.

symbol	meaning	shape
$\xi^{(r)}$	standardized sparse-grid node in $\mathbb{R}^{n_x}$	n_x
w_r	accumulated signed weight after duplicate merge	scalar
m_t, P_t	carried Gaussian mean and covariance after update	n_x, $n_x \times n_x$
m_t^-, P_t^-	predictive Gaussian mean and covariance before update	n_x, $n_x \times n_x$
C_t, C_t^-	covariance factors with $P_t = C_t C_t^\top$, $P_t^- = C_t^- (C_t^-)^\top$	$n_x \times n_x$
$\chi_{t-1}^{(r)}$	previous-state physical cloud point	n_x
$a_t^{(r)}$	transition image of $\chi_{t-1}^{(r)}$	n_x
$\chi_t^{(r)}$	predictive physical cloud point	n_x
$z_t^{(r)}$	observation image of $\chi_t^{(r)}$	n_y
$\bar{z}_t$	predicted observation mean	n_y
S_t	innovation covariance	$n_y \times n_y$
$C_{xz,t}$	state-observation cross-covariance	$n_x \times n_y$
v_t	innovation residual $y_t - \bar{z}_t$	n_y
K_t	Gaussian projection gain	$n_x \times n_y$
$\widehat{\ell}_t^{\mathrm{FSGQ}}$	one-step deterministic approximate log-likelihood increment	scalar

Three implementation conventions are fixed report-wide.

1. **Factor convention.** Every covariance factor is the lower triangular Cholesky factor with positive diagonal, written $C(P) = \mathrm{chol}(P)$, whenever the declared branch is valid.

2. **Solve convention.** Formulas may use S_t^{-1} for compact mathematics, but the implementation contract is to compute all such actions by linear solves against the declared Cholesky factor of S_t, not by forming an explicit inverse.

3. **Noise convention.** Additive process noise enters analytically through the covariance term Q_θ in predictive moments; additive observation noise enters analytically through R_θ in the innovation covariance. This report does not use state augmentation for these additive noises.

When floating-point accumulation produces a covariance that is not exactly symmetric, the report assumes explicit symmetrization by $(P + P^\top)/2$ before branch checking. If the resulting matrix is not positive definite on the declared Cholesky branch, the branch is vetoed rather than silently repaired. Any jitter, clipping, or eigenvalue flooring policy would define a different scalar than the one studied here.

2.2 Gaussian Projection From Moments

Assume the predictive state is represented by

$$X_t^- \sim \mathcal{N}(m_t^-, P_t^-). \tag{2.5}$$

Let

$$Z_t = h_\theta(X_t^-) \tag{2.6}$$

be the noiseless predicted observation. The Gaussian projection needs

$$\bar{z}_t = \mathbb{E}[Z_t], \tag{2.7}$$
$$S_t = R_\theta + \mathbb{E}[(Z_t - \bar{z}_t)(Z_t - \bar{z}_t)^\top], \tag{2.8}$$
$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(Z_t - \bar{z}_t)^\top]. \tag{2.9}$$

Given these moments, define

$$v_t = y_t - \bar{z}_t, \qquad K_t = C_{xz,t} S_t^{-1}, \tag{2.10}$$
$$m_t = m_t^- + K_t v_t, \qquad P_t = P_t^- - K_t S_t K_t^\top. \tag{2.11}$$

Proposition 1 (Affine projection)**.** *Assume the innovation covariance S_t is square and invertible, so the solve or inverse notation below is well defined, and all matrix products are conformable under the stated object map. Among estimators of X_t^- of the form $a + BY_t$, the minimum mean-square estimator under the moments in (2.7)–(2.9) uses*

$$B = C_{xz,t} S_t^{-1}, \qquad a = m_t^- - B\bar{z}_t. \tag{2.12}$$

Storing the resulting mean and covariance as a Gaussian gives (2.11).

Proof. Center $\widetilde{X} = X_t^- - m_t^-$ and $\widetilde{Y} = Y_t - \bar{z}_t$. The squared-error objective for $m_t^- + B\widetilde{Y}$ is

$$J(B) = \mathbb{E}\|\widetilde{X} - B\widetilde{Y}\|^2 = \operatorname{tr}(P_t^-) - 2\operatorname{tr}(BC_{xz,t}^\top) + \operatorname{tr}(BS_tB^\top). \tag{2.13}$$

Differentiating gives $\nabla_B J = -2C_{xz,t} + 2BS_t$. If S_t is nonsingular, the stationary point is $B = C_{xz,t}S_t^{-1}$. Substitution gives the residual covariance $P_t^- - C_{xz,t}S_t^{-1}C_{xz,t}^\top$, which equals (2.11). □

The approximation has two layers. First, the moments (2.7)–(2.9) may be approximated by quadrature. Second, the resulting posterior information is compressed to one Gaussian.

2.3 How This Matches The Jia–Xin–Cheng Gaussian Approximation Block

Jia, Xin, and Cheng first write the Gaussian approximation filter before they introduce sparse grids [1, Section 2.2]. In their additive-noise setting, the prediction moment equations have the same structure as

$$m_t^- = \mathbb{E}[f_\theta(X_{t-1})], \tag{2.14}$$
$$P_t^- = Q_\theta + \mathbb{E}[(f_\theta(X_{t-1}) - m_t^-)(f_\theta(X_{t-1}) - m_t^-)^\top], \tag{2.15}$$

where $X_{t-1} \sim \mathcal{N}(m_{t-1}, P_{t-1})$ under the carried Gaussian approximation. The update moment equations have the structure

$$\bar{z}_t = \mathbb{E}[h_\theta(X_t^-)], \tag{2.16}$$

$$S_t = R_\theta + \mathbb{E}[(h_\theta(X_t^-) - \bar{z}_t)(h_\theta(X_t^-) - \bar{z}_t)^\top], \tag{2.17}$$
$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(h_\theta(X_t^-) - \bar{z}_t)^\top]. \tag{2.18}$$

Then the Gaussian projection update is

$$K_t = C_{xz,t} S_t^{-1}, \tag{2.19}$$
$$m_t = m_t^- + K_t(y_t - \bar{z}_t), \tag{2.20}$$
$$P_t = P_t^- - K_t S_t K_t^\top. \tag{2.21}$$

These equations are not yet sparse-grid equations. They are the Gaussian moment closure equations that any deterministic Gaussian quadrature lane must eventually supply. The next chapter now turns from that formal target to the concrete low-dimensional construction of the sparse-grid cloud that will later be inserted into the recursion.

3 Low-Dimensional Construction Of FixedSGQF

The sparse-grid rule is easiest to understand before it is inserted into the full filtering recursion. The present chapter therefore begins with low-dimensional constructions. It starts from one nonlinear Gaussian moment in one coordinate, lifts that rule to a tensor-product cloud in two coordinates, and then asks how the same construction becomes too expensive in higher dimension and how the sparse-grid combination reduces that cost without changing the underlying standardized-coordinate logic.

3.1 One nonlinear Gaussian moment as the basic quadrature problem

Start with a scalar Gaussian input and a simple nonlinear observation map. Let

$$X \sim \mathcal{N}(m, P), \qquad h(X) = X^2. \tag{3.1}$$

Suppose we want the Gaussian-projection observation mean

$$\bar{z} = \mathbb{E}[h(X)] = \mathbb{E}[X^2]. \tag{3.2}$$

Write $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, 1)$ and $C^2 = P$. Then

$$\bar{z} = \int_{\mathbb{R}} (m + C\xi)^2 \phi(\xi)\, \mathrm{d}\xi. \tag{3.3}$$

Once the state has been standardized, the moment problem becomes a standard-normal weighted integral. The quadrature rule is therefore not an extra modeling choice layered on top of nowhere. It is the numerical approximation to the Gaussian expectation in (3.3).

In this scalar example, a three-point symmetric rule already reproduces the first few Gaussian moments exactly, which is why rules such as $\{-\sqrt{3}, 0, \sqrt{3}\}$ appear naturally later. The next step is to make that one-dimensional rule explicit and then to see what happens when the same rule is lifted into more than one coordinate.

3.2 Tensor-product growth in two dimensions

Now move from one coordinate to two. Use a simple two-dimensional Gaussian state,

$$X \sim \mathcal{N}(m, P), \qquad X \in \mathbb{R}^2, \tag{3.4}$$

and imagine either a two-dimensional linear-Gaussian state-space step or a slightly nonlinear observation moment built from that same Gaussian input. The point of this example is geometric rather than inferential: it shows how the same standardized cloud is placed and why tensor products grow quickly.

Let $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, I_2)$. If each standardized coordinate uses the three-point rule $\{-\sqrt{3}, 0, \sqrt{3}\}$, then the direct tensor product uses all pairs

$$(\xi_1, \xi_2) \in \{-\sqrt{3}, 0, \sqrt{3}\} \times \{-\sqrt{3}, 0, \sqrt{3}\}, \tag{3.5}$$

so already in two dimensions the rule has $3^2 = 9$ points. In three dimensions the same construction has $3^3 = 27$ points; in ten dimensions it has 3^{10} points. The tensor-product rule is therefore transparent but already expensive enough to motivate a more economical construction.

At this stage there is still no sparse-grid machinery. Nothing has yet been combined or pruned. We are only taking one one-dimensional rule and using it in each coordinate. The tensor-product construction should therefore be read as the rectangular standardized cloud that later sparse-grid ideas will modify, not as a separate competing method.

What does the symbol F mean? When the note later writes expressions such as $I_{\mathbf{i}}[F]$ or $A_{b,L}[F]$, the symbol F is just the multivariate integrand being averaged under the Gaussian weight. In one dimension the same role is often played by ψ. In several coordinates we simply switch to $F(\xi)$ to remind the reader that the function may depend on many variables. A concrete example to keep in mind is

$$F(\xi_1, \xi_2, \xi_3) = e^{\xi_1 \xi_2 \xi_3},$$

which will be reused later to illustrate what a low sparse-grid level can and cannot see.

3.3 One-Dimensional Rules And Their Tensor Products

The scalar example above shows why a Gaussian expectation becomes a standard-normal weighted integral after standardization. The next step is to fix a concrete family of one-dimensional rules and then lift those rules into multiple coordinates by tensor products.

What is a standard-normal Gauss–Hermite rule? A standard-normal Gauss–Hermite quadrature rule is a deterministic weighted-sum approximation to a standard-normal integral,

$$\int_{\mathbb{R}} \psi(\xi)\phi(\xi)\,\mathrm{d}\xi,$$

chosen so that the rule is exact for as many low-degree polynomials as possible for its number of points. One picks nodes $\zeta_{\ell,a}$ and weights $\omega_{\ell,a}$, evaluates the integrand at those nodes, and forms the weighted sum

$$\sum_{a=1}^{s_\ell} \omega_{\ell,a}\psi(\zeta_{\ell,a}).$$

Once a Gaussian moment has been standardized to the coordinate ξ, standard-normal GHQ gives a canonical deterministic way to approximate that moment.

Which concrete family is being used in this note? At the teaching level, the easiest way to read the symbols is the following. In this note, the default BayesFilter family is the standard-normal GHQ ladder:

$I_1 = 1$-point standard-normal GHQ, $I_2 = 3$-point standard-normal GHQ, $I_3 = 5$-point standard-normal G

So the level label is a rule-family label, not a polynomial degree. Level 2 does not mean "degree 2 polynomial." It means "the second low-level member of the chosen one-dimensional GHQ family," which in this note is the familiar symmetric three-point rule. The later formal policy statement (**??**) records the same choice more precisely.

Where does the level-1 rule $I_1 : (0, 1)$ come from? It is the simplest one-dimensional standard-normal rule: one node at the mean $\xi = 0$ with total weight 1. In the notation of the general one-dimensional rule, this means $s_1 = 1$, $\zeta_{1,1} = 0$, and $\omega_{1,1} = 1$, so

$$I_1[\psi] = \psi(0).$$

This rule already integrates constants exactly, because

$$\int_{\mathbb{R}} 1 \cdot \phi(\xi)\, \mathrm{d}\xi = 1.$$

So $I_1 : (0, 1)$ is the natural level-1 base rule: evaluate at the center of the standardized Gaussian and assign it the full mass.

3.3.1 A fully worked one-dimensional toy rule

Take the symmetric three-point ansatz

$$\{(-a, \omega), (0, \omega_0), (a, \omega)\}. \tag{3.6}$$

To match the first relevant Gaussian moments, require

$$\omega_0 + 2\omega = 1, \tag{3.7}$$
$$2\omega a^2 = 1, \tag{3.8}$$
$$2\omega a^4 = 3. \tag{3.9}$$

Dividing (3.9) by (3.8) gives $a^2 = 3$. Then $\omega = 1/6$ and $\omega_0 = 2/3$. So the rule becomes

$$\left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{3.10}$$

This solved three-point rule is exactly the later level-2 rule I_2 used in the two-dimensional and three-dimensional examples. In other words, I_2 is not a new assumption introduced later; it is simply shorthand for the symmetric three-point standard-normal rule obtained from (3.7)–(3.9).

3.3.2 Lifting the rule to two dimensions

The multi-index enters only because different coordinates may use different one-dimensional rule levels. Its role is bookkeeping, not new mathematics. For example:

- $(1, 1)$ means "use level 1 in coordinate 1 and level 1 in coordinate 2,"

- $(1, 2)$ means "use level 1 in coordinate 1 and level 2 in coordinate 2,"
- $(2, 1)$ means "use level 2 in coordinate 1 and level 1 in coordinate 2,"
- $(2, 2)$ means "use level 2 in both coordinates."

Formally, for a multi-index $\mathbf{i} = (i_1, \ldots, i_b)$, the corresponding tensor rule is

$$I_{\mathbf{i}}[F] = (I_{i_1} \otimes \cdots \otimes I_{i_b})[F]. \tag{3.11}$$

This means: apply the one-dimensional rule of level i_1 in the first coordinate, the one-dimensional rule of level i_2 in the second coordinate, and so on, then take the full Cartesian product of the resulting nodes and weights.

In the concrete two-dimensional case, take $b = 2$ and use

$$I_1 : (0, 1), \qquad I_2 : \left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{3.12}$$

Then the rule $(1, 1)$ gives only $(0, 0)$. The rule $(1, 2)$ gives the three vertical points $(0, -\sqrt{3})$, $(0, 0)$, and $(0, \sqrt{3})$. The rule $(2, 1)$ gives the three horizontal points $(-\sqrt{3}, 0)$, $(0, 0)$, and $(\sqrt{3}, 0)$. If both coordinates use level 2, then the full tensor product rule $(2, 2)$ gives the full nine-point grid

$$(-\sqrt{3}, -\sqrt{3}), (-\sqrt{3}, 0), (-\sqrt{3}, \sqrt{3}), (0, -\sqrt{3}), (0, 0), (0, \sqrt{3}), (\sqrt{3}, -\sqrt{3}), (\sqrt{3}, 0), (\sqrt{3}, \sqrt{3}), \tag{3.13}$$

with weights given by products of the one-dimensional weights.

The tensor-product rule is therefore simple but already expensive: if each coordinate uses s nodes, then the point count is s^b. In two dimensions with three points per coordinate, one already needs nine points; in three dimensions the same direct construction would require twenty-seven. The next step is therefore not to abandon the standardized-coordinate picture, but to reduce the cost of that picture.

3.4 Three-dimensional preview: why sparse grids help

Use the same one-dimensional rules as before:

$$I_1 : (0, 1), \qquad I_2 : \left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{3.14}$$

If all three coordinates use level 2, then the full tensor-product rule $(2, 2, 2)$ places

$$3 \times 3 \times 3 = 27 \tag{3.15}$$

standardized points. The sparse-grid idea begins by asking whether all those point combinations are equally important at low level.

Now compare that with the sparse-grid band for $b = 3$ and $L = 2$. This is a different use of the word "level." The tensor-product label $(2, 2, 2)$ means "use the one-dimensional level-2 rule in every coordinate." By contrast, the sparse-grid level $L = 2$ means: combine all tensor rules whose total index lies in the admissible band for $(b, L) = (3, 2)$. So the sparse-grid construction does not keep the full 27-point tensor rule $(2, 2, 2)$. Instead, it mixes several lower-total-index tensor rules with signed coefficients.

The allowed multi-indices satisfy

$$2 \leq |\mathbf{i}| \leq 4. \tag{3.16}$$

Since every coordinate level is at least 1, the only admissible three-dimensional multi-indices are

$$(1,1,1), \qquad (1,1,2), \qquad (1,2,1), \qquad (2,1,1). \tag{3.17}$$

So the sparse-grid rule does not use the full level-2 tensor product $(2,2,2)$. It instead combines one one-point tensor rule and three one-axis three-point tensor rules. This is the first low-dimensional picture of why sparse grids help: they preserve the same standardized-coordinate logic while paying first for lower-order interaction structure rather than for every corner of the full tensor product.

4 Filtering Value Path And Worked Example

Fix a standardized sparse-grid cloud

$$\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M}, \qquad \xi^{(r)} \in \mathbb{R}^{n_x}, \quad w_r \in \mathbb{R}. \tag{4.1}$$

The cloud is frozen before likelihood evaluation. The value path has two quadrature stages at each time.

What this section gives the implementer. This section is the authoritative one-step value recursion. A value routine constructed from this note should take the current carried Gaussian, current observation, fixed cloud, and declared model objects as input, then return either an accepted one-step increment with updated (m_t, P_t) or a branch failure record.

4.1 Prediction Stage

Let $P_{t-1} = C_{t-1}C_{t-1}^{\top}$ on the declared square-root branch. Place the previous Gaussian cloud:

$$\chi_{t-1}^{(r)} = m_{t-1} + C_{t-1}\xi^{(r)}. \tag{4.2}$$

Propagate each point through the transition map,

$$a_t^{(r)} = f_\theta(\chi_{t-1}^{(r)}), \tag{4.3}$$

and form the predictive moments

$$m_t^- = \sum_{r=1}^{M} w_r a_t^{(r)}, \tag{4.4}$$

$$P_t^- = Q_\theta + \sum_{r=1}^{M} w_r (a_t^{(r)} - m_t^-)(a_t^{(r)} - m_t^-)^{\top}. \tag{4.5}$$

If the symmetrized P_t^- fails the declared Cholesky branch, the value path returns a predictive-branch failure record rather than a scalar.

4.2 Observation Stage

Assume the predictive branch is valid, so that

$$P_t^- = C_t^-(C_t^-)^{\top}. \tag{4.6}$$

Place the predictive cloud,

$$\chi_t^{(r)} = m_t^- + C_t^- \xi^{(r)}, \tag{4.7}$$

propagate it through the observation map,

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}), \tag{4.8}$$

and compute the observation moments

$$\bar{z}_t = \sum_{r=1}^{M} w_r z_t^{(r)}, \tag{4.9}$$

$$S_t = R_\theta + \sum_{r=1}^{M} w_r (z_t^{(r)} - \bar{z}_t)(z_t^{(r)} - \bar{z}_t)^\top, \tag{4.10}$$

$$C_{xz,t} = \sum_{r=1}^{M} w_r (\chi_t^{(r)} - m_t^-)(z_t^{(r)} - \bar{z}_t)^\top. \tag{4.11}$$

Then define

$$v_t = y_t - \bar{z}_t, \tag{4.12}$$

$$K_t = C_{xz,t} S_t^{-1}, \tag{4.13}$$

$$m_t = m_t^- + K_t v_t, \tag{4.14}$$

$$P_t = P_t^- - K_t S_t K_t^\top. \tag{4.15}$$

The one-step deterministic approximate log-likelihood increment is

$$\widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right\}. \tag{4.16}$$

The accumulated scalar is the sum of these increments over time.

4.3 Worked One-Step Example With A Numeric Oracle

Use the scalar nonlinear model

$$X_1 = \beta X_0 + \eta_1, \qquad \eta_1 \sim \mathcal{N}(0, Q), \qquad Y_1 = X_1^2 + \varepsilon_1, \qquad \varepsilon_1 \sim \mathcal{N}(0, R), \tag{4.17}$$

with

$$X_0 \sim \mathcal{N}(m_0, P_0), \qquad m_0 = 0.5, \qquad P_0 = 1, \qquad \beta = 1, \qquad Q = 4, \qquad R = 1, \qquad y_1 = 3. \tag{4.18}$$

Use the three-point level-2 cloud

$$\xi \in \{-\sqrt{3}, 0, \sqrt{3}\}, \qquad w \in \{1/6, 2/3, 1/6\}.$$

Then

$$m_1^- = \beta m_0 = 0.5, \qquad P_1^- = \beta^2 P_0 + Q = 5, \tag{4.19}$$

$$C_1^- = \sqrt{5}, \qquad \chi_1^{(r)} \in \{0.5 - \sqrt{15},\ 0.5,\ 0.5 + \sqrt{15}\}. \tag{4.20}$$

The observation values are

$$z_1^{(r)} = (\chi_1^{(r)})^2 \in \left\{ \frac{31}{2} - \sqrt{15},\ \frac{1}{4},\ \frac{31}{2} + \sqrt{15} \right\}. \tag{4.21}$$

From these,

$$\bar{z}_1 = 2.5 = \frac{5}{2}, \qquad S_1 = 5.375 = \frac{43}{8}, \tag{4.22}$$

$$C_{xz,1} = 1.25 = \frac{5}{4}, \qquad v_1 = 0.5 = \frac{1}{2}, \tag{4.23}$$

$$K_1 = \frac{10}{43} \approx 0.2325581395, \qquad m_1 = \frac{53}{86} \approx 0.6162790698, \tag{4.24}$$

$$P_1 = \frac{165}{172} \approx 0.9593023256. \tag{4.25}$$

The one-step deterministic approximate log likelihood is

$$\widehat{\ell}_1^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{ \log S_1 + v_1^2 S_1^{-1} + \log(2\pi) \right\} \approx -1.7830736342. \tag{4.26}$$

This is the value-path oracle used later for both implementation checking and same-branch derivative verification.

5 Same-Scalar Branch Contract And Analytical Gradient

The branch record $\mathcal{B}$ freezes both the value-defining structural choices and the derivative-support metadata needed to differentiate the same scalar on that branch. The value scalar itself depends only on the former, while the derivative-support objects matter only when the gradient lane is requested. A fixed-SGQF branch is the tuple

$$\begin{aligned} \mathcal{B} = (&y_{1:T},\ \text{observation preprocessing},\ m_0(\theta), P_0(\theta), \dot{m}_0, \dot{P}_0, \\ &\mathcal{C},\ \text{one-dimensional rules},\ \text{duplicate-merge tolerance},\ \text{zero-weight rule},\ \text{node ordering}, \\ &C(P) = \mathrm{chol}(P),\ \text{symmetrize-then-veto rule},\ \epsilon_S, \epsilon_P, \\ &f_\theta, h_\theta, Q_\theta, R_\theta,\ D_x f_\theta, \partial_i f_\theta, D_x h_\theta, \partial_i h_\theta, \dot{Q}_\theta, \dot{R}_\theta). \end{aligned} \tag{5.1}$$

The fixed scalar is

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta; \mathcal{B}) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta; \mathcal{B}), \tag{5.2}$$

where every term is computed by (4.2)–(4.15). Changing the grid level, merge tolerance, node ordering, factor branch, or veto rule changes the scalar. The derivative formulas below therefore cover the open branch where the declared branch remains valid without repair.

Plain-language branch rule. The branch identity freezes structure, not numerical values. When θ changes inside a same-scalar finite-difference check, the moments, gains, and likelihood contributions are recomputed, but they must be recomputed by the same cloud, the same merge/order policy, the same factor family, and the same realized acceptance path through the branch logic. This is the operational meaning of "same scalar" for implementation review.

5.1 Analytical Gradient Of The Fixed Scalar

The derivative chapter follows the same order as the value chapter. Differentiate cloud placement, transition images, predictive moments, observation images, innovation statistics, and finally the innovation log-likelihood increment, all while holding the branch record fixed.

Why this derivative is operationally delicate. The scalar is deterministic only on a declared branch. The derivative is therefore not merely the formal derivative of some symbolic quadrature formula. It is the derivative of the declared deterministic scalar evaluated on the same branch. If a nearby perturbation changes the merge rule, factor branch, or veto path, the finite difference has left the scalar being differentiated.

5.1.1 The Story Of The Derivative

The derivative propagation is easiest to read in the same order as the value path: place the cloud, push it through the transition, form predictive moments, place again, push through the observation, form innovation statistics, then accumulate the score. The detailed formulas and sensitivity identities from the current report remain unchanged and are retained below as the formal derivative contract.

5.1.2 Square-Root Branch

The declared covariance factor is the lower-triangular Cholesky factor. Its matrix differential convention, derivative identities, and subsequent sensitivity propagation rules are the same as in p44 and remain the authoritative branch specification here.

5.1.3 Prediction Sensitivities

The prediction sensitivities are obtained by differentiating the cloud placement, transition images, predictive mean, and predictive covariance under the fixed cloud and branch record. The formulas retained from the current report remain unchanged and continue to define the same-scalar derivative contract.

5.1.4 Observation Sensitivities

The observation sensitivities are obtained by differentiating the predictive cloud placement, observation images, predicted observation mean, innovation covariance, and cross-covariance under the same branch.

5.1.5 Innovation Score

The innovation score is the derivative of the one-step Gaussian innovation log likelihood under the declared branch. The detailed formulas from the current report remain the operational score definition.

5.1.6 Posterior Sensitivity Propagation

Once the score, gain, and covariance sensitivities have been computed, propagate the posterior mean and covariance sensitivities in the same order as the value path. The report's existing derivative formulas remain the authoritative implementation contract.

5.2 One Boxed Mathematical Algorithm

The boxed algorithm remains the chapter summary of the full same-scalar value and gradient recursion. In p45 it remains at the end of the derivative chapter so that it reads as a culmination rather than as a competing chapter line.

6 Verification And Validation

The fixed-SGQF lane is only useful if value, branch, and derivative claims can be verified on the same scalar. This chapter therefore gathers the finite-difference check, the worked toy traces, and the diagnostic test models into one verification ledger.

6.1 Finite-Difference Same-Scalar Check

The same-scalar finite-difference check asks one narrow question: if the branch identity remains unchanged under a small parameter perturbation, does the analytical gradient agree with the finite difference of the same declared scalar? The detailed formulas and thresholds from the current report remain unchanged; the point of the present reorganization is only to keep them in one verification chapter.

Implementation-facing FD pseudocode summary. The finite-difference routine should perturb parameters only inside the same branch, re-evaluate the fixed scalar on that same branch identity, and reject any comparison that silently changes the structural scalar.

6.2 Diagnostics, Accuracy, Memory, And Performance Tests

The diagnostic chapter keeps the same hierarchy of checks as before, but now reads as one verification ledger rather than as an independent companion note. Use the linear-Gaussian recovery case to check exact collapse, the scalar quadratic case to check low-order moment correctness, the cloud-sensitive nonlinear case to check the effect of cloud placement, the block-local high-dimensional case to test the intended sparse-grid advantage, and the hard all-coordinate interaction case to mark the lane's scientific boundary.

Model A: linear-Gaussian recovery. This is the exact-collapse test. FixedSGQF should match the Kalman recursion when the model is linear-Gaussian.

Model B: scalar quadratic observation. Use (6.1) as the smallest nonlinear moment test that still admits a transparent oracle.

$$X \sim \mathcal{N}(0, P), \qquad Y = X^2 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, R). \tag{6.1}$$

Model C: cloud-sensitive two-dimensional nonlinear observation. This is the first test where cloud geometry, not only algebraic consistency, matters.

Model D: block-local high-dimensional nonlinear model. This is the first intended operating regime for a moderate sparse-grid level.

Model E: deliberately hard all-coordinate interaction. This is the regime in which the Gaussian-surrogate lane becomes too narrow and the note should fail honestly rather than overclaim.

Model F: Jia–Xin–Cheng orbit-style benchmark. This preserves the source-linked large-scale benchmark lane already present in the report.

7 Adaptive Sparse Grids As Grid Design

Adaptive sparse grids remain relevant in this report only as an offline grid-design remark. The present fixed-scalar derivative contract does not differentiate a live adaptive selection rule. So adaptation belongs here as a late remark on grid design, not as a competing main-line method.

8 Relation To Neighboring High-Dimensional Filtering Proposals

The present lane should now be positioned only after its object, cloud, recursion, derivative, and verification obligations are already clear. The main comparison question is not whether FixedSGQF dominates every neighboring method. It is where this deterministic Gaussian-surrogate lane sits among other serious high-dimensional proposals.

The UKF comparison should now be read mainly by cross-reference back to the low-dimensional construction chapter. The key message has already been stated there: low-level SGQF can look UKF-like geometrically and may even share final weights in special symmetric-axis cases, but the sparse-grid construction is assembled from one-dimensional rules, active tensor-rule selection, and Smolyak recombination rather than from one directly specified sigma-point formula.

The remaining positioning arguments against EKF, dense GHQF, live adaptive sparse grids, particle methods, and density-based tensor methods are preserved from p44, but now read as late synthesis rather than as repeated restarts of the main lane claim.

9 Conclusion

FixedSGQF is a deliberately narrow high-dimensional filtering lane. It does not attempt to preserve a full non-Gaussian posterior law. It carries one Gaussian surrogate, computes the required nonlinear moments by a fixed sparse-grid cloud, and differentiates the same declared deterministic scalar on a frozen branch. That narrowness is the point. It is what makes the method reproducible, auditable, and mathematically explicit enough to support implementation and verification.

The note therefore closes with a bounded claim. It does not argue that FixedSGQF is the best nonlinear filter overall. It argues that, when a deterministic Gaussian-surrogate lane with a same-scalar analytical gradient is the desired object, the fixed sparse-grid construction gives a coherent and technically well-specified way to build that lane.

A Implementation Contract

The implementation contract material from p44 is retained here as appendix-style technical support rather than as part of the main chapter line. The detailed input-output contracts, failure conventions, and runtime invariants remain unchanged and continue to govern any implementation built from the note.

B End-To-End Mathematical Algorithm

The end-to-end algorithm remains as appendix material so that the main text can carry the mathematical argument while the appendix records the full execution ordering explicitly.

C Where Each Construction Comes From

The source map remains useful as an audit appendix linking the BayesFilter construction back to the underlying literature and internal specialization choices.

D Notation And Formula Inventory

The notation and formula inventory remains as late technical support rather than as a main-text interruption.

References

[1] Bin Jia, Ming Xin, and Yang Cheng. Sparse-grid quadrature nonlinear filtering. *Automatica*, 48 (2):327–341, 2012. doi: 10.1016/j.automatica.2011.08.057.